

Finding a watermark in model-written DNA without knowing where it starts

Kimon Antonios Provatas^{1,*} Christos Galanopoulos^{1,*} Ilias Georgakopoulos-Soares^{1,†}

¹The University of Texas at Austin, Austin, Texas, USA

*These authors contributed equally. †Correspondence to I.G.-S.

Abstract

Genomic language models now write DNA that reads like the real thing, and a sequence carries no record of where it came from. A watermark added while the model writes would let the holder of a secret key recognise the model’s output later, but DNA makes this harder than text. A DNA sequence arrives with no punctuation: the reader does not know which base the model’s output starts at, so it does not know how to group the bases into the words the model actually wrote, and the sequence may have been reverse-complemented or pasted inside a longer stretch of DNA.

We add the SynthID tournament watermark to two genomic language models that write DNA six bases at a time, Carbon-500M and GENERator-v2 1.2B, and we ask two questions of each. Does the watermark change what the model writes? And can a verifier find it without being told where the model’s output begins? We measure the first against each model’s own likelihood and a pre-specified panel of sequence statistics, and find no difference we can detect in either model. For the second, we give the verifier nothing but the DNA and the key, and let it search both strands, every starting base, and four sequence lengths, charging the full cost of that search to a single answer for each sequence. In both models it found the watermark in all 384 held-out marked reads, unedited and after any one base was substituted, inserted, or deleted, while unmarked reads and reads checked with the wrong key produced at most one positive each. The margin is wide enough that paying for the search costs almost nothing, and an inserted or deleted base, which shifts every downstream word, is absorbed by the search rather than by the statistic. That the same implementation behaves this way in two models of different size and training supports the portability of the implementation; it does not show that the two models are equivalent. These are statistical results about provenance. They do not speak to biological function, and they do not measure how hard a deployed key would be to recover. They also rest on one execution per model, on hardware other than the machine named for their replication, and that replication is outstanding.

1 Introduction

A DNA sequence carries no record of its own origin. Once it is copied out of the software that produced it, nothing in the letters says whether a person designed it, an experiment recovered it, or a model wrote it. That was a minor problem while generative models for DNA were weak. It is no longer minor. Models trained on genomes now write coding sequences, regulatory elements, and whole synthetic loci with enough fluency that their output is difficult to separate from natural sequence by eye or by simple statistics [24, 27, 3]. The same fluency has made provenance a stated concern for the people who screen and synthesise DNA, who have asked for records that survive from design software through to the ordered oligonucleotide [2].

One way to leave such a record is to hide a signal in the sequence while the model writes it, so that whoever holds a secret key can later check a piece of DNA and get an answer without needing

the model, the prompt, or any log of the generation. This is what watermarking does for text. Carrying it to DNA is not a matter of changing the alphabet. Text arrives with spaces, so a verifier can recover the tokens the model produced. A DNA sequence arrives as an unbroken run of letters. A model that writes six bases at a time leaves a signal that only makes sense once the letters are regrouped into those same six-base words, and there are six ways to group them. The verifier is usually not told which is right, because it is usually not told where the model’s output begins. The sequence may also arrive reverse-complemented, or embedded in a longer stretch of natural DNA. Each of those unknowns has to be searched, and every search costs statistical evidence, so a watermark that is easy to find when the boundary is known can vanish once it is not.

This paper takes one watermark, two genomic language models, and one verifier, and asks whether that combination survives the honest version of the problem. We add the SynthID tournament watermark [8] to Carbon-500M [3] and to GENERator-v2 1.2B [22], two models that write DNA in six-base tokens and differ in size, training corpus, and tokenizer construction, and we ask two questions of each. First, does adding the watermark change what the model writes, measured by the model’s own likelihood and by a panel of sequence statistics fixed before any comparison was run? Second, can a verifier that holds only the DNA and the key find the watermark while paying, in full and in advance, for the search over strand, starting base, and length?

The answers are the same in both models. We could not measure any change in what either model writes, and the verifier found the watermark in every held-out watermarked read from either model, unedited and after any single substituted, inserted, or deleted base, with unmarked and wrong-key reads behaving as the test predicts. Running one implementation through two models is a test of the implementation, not a comparison of the models: it tells us the construction and the verifier carry across a change of model, and it says nothing about which model writes better DNA. The evidence margin turns out to be so large that the price of the search is almost free, and the search itself repairs the damage an insertion or deletion does to the six-base grouping.

We are careful about what those answers cover. We separate four kinds of statement and never mix them: identities of the construction, assumptions about the key, statistical measurements, and sequence statistics that stand in for quality. Nothing here shows that the DNA is biologically functional, viable, or safe. Nothing here shows that a deployed secret key resists recovery. To keep the argument readable, Table 1 fixes one word and one symbol for each idea, and the rest of the paper uses only those.

2 Background

2.1 Marks written into DNA before models wrote DNA

Hiding a message in DNA is an old idea, and it predates generative models by two decades. The first synthetic bacterial genome carried four watermark sequences inserted at positions chosen not to disturb the cell, encoding names and an email address, so that the synthetic chromosome could be told apart from its natural counterpart [12]. Schemes such as DNA-Crypt made the same idea systematic, encoding an encrypted payload into synonymous codon positions and adding error correction so that the message survives a few mutations [16]. A mark of this kind can be made to survive biology as well as software: hiding text in the synonymous codon choices of an open reading frame leaves the encoded protein untouched and ties the message to the frame even when it is taken out of its context, and one such marked viral gene was recovered with the message intact after 136 days of replication [23].

These are insertions. Something extra is added to a sequence that was going to exist anyway, and the mark is a foreign object inside it. That is workable when a person controls the design and

Table 1: Words and symbols used in this paper. Each idea has one name; no synonyms appear elsewhere in the text, the tables, or the figures.

Word or symbol	Meaning
model	either of the two used here, each restricted to its 4,096 six-base DNA tokens
Carbon	Carbon-500M
GENERator	GENERator-v2-eukaryote-1.2b-base
token	one DNA word of six bases
prompt	384 bases of real DNA given to the model
continuation	the 3,072 bases the model writes
read	what the verifier receives: prompt and continuation, $N = 3,456$ bases
mark	the SynthID tournament watermark
key, k	the secret used both to write the mark and to read it
published key	a key released with this study so the runs can be reproduced
mark bit, g	a keyed value of 0 or 1 for one candidate token in one context
layer, m	one keyed round of the tournament; $m = 30$ layers are applied
context	the four tokens written just before the current one
p	the model’s probability for a candidate token
p_k	the probability the sampler actually uses, after the m layers
window	a stretch of the read that the verifier scores
L	window length in bases; $L \in \{384, 768, 1,536, 3,072\}$
M	windows searched in one read; $M = 16,136$ for $N = 3,456$
n	mark bits scored inside a window
S	how many of those n bits are 1
P_{win}	the chance probability of one window
P_{read}	the chance probability of the whole read, after paying for the search
window strength	$-\log_{10} P_{\text{win}}$, so larger means stronger evidence
recall	share of marked reads in which the mark was found
false-positive rate	share of control reads in which the mark was found
α	the target false-positive rate for one read; $\alpha = 0.01$
ordinary	sampled straight from p , with no key
draw	one of the two continuations generated per prompt in each model

can pick a harmless location, and it fails in the setting we care about, because there is no separate design step: the model writes the sequence, and there is no spare region to place a payload in. It is also brittle in a different way. An inserted marker is a fixed string, so anyone who learns it can find it, remove it, or forge it.

2.2 Watermarking what a language model writes

Watermarking a language model works differently. Nothing is inserted afterwards; instead the sampling step itself is nudged, so that the output is drawn from a distribution that a keyed test can recognise but an outside reader cannot. The first widely used scheme splits the vocabulary at each step into a pseudorandom green list and red list and pushes probability toward the green list, then detects the mark by counting green tokens [20]. It is simple and strong, and it moves the output distribution, which shows up as a quality cost when the model is confident.

A second line of work removes that cost. If the keyed randomness is fresh at every step, the sampling rule can be arranged so that, averaged over keys, the output distribution is exactly the model’s. Cryptographic constructions make this precise and show that, without the key, watermarked output is computationally indistinguishable from ordinary output [6]. A practical family reaches the

same property by mapping a key-derived random sequence into a sample through inverse-transform or exponential-minimum sampling, and by aligning text against that sequence at detection time, which also buys robustness to editing and paraphrase [21]; the exponential-minimum rule is usually credited to an earlier proposal of Aaronson and Kirchner [1]. A second branch of the same family reaches the property by reweighting the model’s distribution directly, rather than by mapping keyed randomness into a sample, and recovers the model’s distribution once the average over keys is taken [17, 29]; the construction we use belongs to this branch. A third design gives up the secret instead: a publicly verifiable signature is embedded by rejection sampling, so anyone can run the detector, at the cost of needing enough entropy in the output to carry the signature [10]. There is a limit to how far robustness can be pushed: an attacker who can paraphrase a text at quality has, in general, enough power to remove any watermark from it, so robustness claims are always claims about a stated edit budget rather than about attackers in general [30]. That budget is not a formality. Paraphrasers tuned against a particular scheme evade every watermark they were tested on [9], recursive paraphrasing lowers detection rates at modest cost in quality [26], and querying a watermarked model reveals enough of its rule to both forge and strip the mark [19]. Keys are attacked as well as marks: the keyed sequence of the distortion-free family has been recovered by adaptive prompting [25], and reusing one key across outputs biases the very distribution that family sets out to preserve [28]. Some of this is answerable in turn—forged text can be told apart from genuinely marked text after the fact [13]—and none of it is measured here, for reasons we state with the other limits.

2.3 SynthID

The construction we use is the binary tournament of SynthID-Text, developed at Google DeepMind [8]. At each generation step, a secret key together with a short window of recently generated tokens determines a set of hidden binary values, one per candidate token per layer. The candidates are then run through a tournament: at each layer, competitors are paired and the one whose hidden value is 1 is preferred, so probability flows toward candidates that happen to carry 1s. Because the hidden values are redrawn for every new context, the tournament does not favour any particular token overall; averaged over fresh keyed functions it leaves the model’s next-token distribution unchanged. Detection is a count: recompute the hidden values from the output alone and ask whether too many of them are 1. SynthID is the backbone of this work because two of its properties matter more in DNA than they do in text. Its hidden values depend on recent tokens rather than on absolute position, so the mark travels with a sequence that is moved, cropped, or pasted somewhere else; and its detector needs only the output and the key, not the model, the prompt, or the probabilities, which is exactly the situation a verifier of a DNA sequence is in. It has also been evaluated at scale in a live text deployment, which makes it a reasonable place to start rather than a construction invented for this paper.

2.4 Genomic language models, and what DNA changes

Genomic models have moved quickly from representation learning to generation [7, 24]. Several of the recent generative models, including both used here, do not read DNA one base at a time; they tokenise it into non-overlapping six-base words, which shortens sequences and lets the model reach long genomic contexts [27, 3, 22]. Recent models in this family also reconcile that coarse tokenisation with single-base resolution by marginalising the token distribution down to per-base probabilities [22]. That choice of six-base tokenisation is what makes the watermarking problem different. In text, whitespace and subword vocabularies let a verifier recover the token sequence

the model produced. In DNA the tokens are invisible: the six-base grouping depends entirely on where counting starts, so a verifier that does not know the boundary must consider all six groupings. Reverse complementation doubles that, and cropping or embedding a generated stretch in a longer sequence multiplies it again. A test that reports the best result over all of those attempts without paying for them is not a test at all, which is why the price of the search is central to what follows.

One practical consequence is worth stating before the methods. At least one released checkpoint in this family samples base by base from its marginalised distribution, even though its paper presents that marginalisation as a training objective [11]. What a model samples is therefore a property of the code that was run rather than of the paper describing it, which is why both models here are pinned to a released revision and driven through a distribution we specify ourselves.

Watermarking for DNA models is new. Recent work embeds marks in designed coding sequences through synonymous codon substitution and propagates them from DNA into the translated protein, which is a different and complementary goal: it aims at function-preserving marks in coding regions, evaluated by detection accuracy on the designed sequences [31]. The same reweighting idea has been carried into protein design, where a keyed mark lets a synthesiser verify locally that a sequence came from an authorised user without reporting the sequence to a central database [4]. Our question is narrower and, we think, prior to it. We do not assume the generated region is coding, we do not assume it can be aligned to anything, and we do not assume the verifier knows where it starts. We take an existing watermark whose distributional behaviour is already understood, put it inside a genomic model, and measure what a verifier can honestly conclude from a bare stretch of DNA and a key.

3 Methods

3.1 Model and sampling

Generation used two models, each at one pinned revision of its released checkpoint: Carbon-500M [3, 18] and GENERator-v2-eukaryote-1.2b-base [22, 11]. Both are decoder-only genomic language models that tokenise DNA into non-overlapping six-base words, and they differ in size, training corpus, and tokenizer construction. Every stage below was run separately and identically for each model, and no number is pooled across them.

At each step the model’s logits were restricted to the 4,096 canonical DNA tokens and normalised once, which for GENERator means its direct canonical token distribution rather than the base-by-base path its released code uses by default. Temperature was 1.0 and no top- k or top- p truncation was applied. The ordinary arm samples from that distribution directly and uses no key. The marked arm applies the tournament described below and then samples.

Generation ran on GPU hardware. Model scoring ran on CPU for Carbon and on GPU for GENERator, and detection ran on x86-64 CPUs for both. That is not the machine this study names for its own replication, which we state with the other limits.

3.2 Writing the mark

At each step the model gives a probability p to each of 4,096 tokens. The key, the four tokens of context, and each candidate token together fix that candidate’s mark bit g , which is 0 or 1. Mark bits are derived with HMAC-SHA-256 over the key, a fixed domain label, the context, and the candidate token. The tournament formula, the number of layers, the context width, and the repeated-context rule follow the public reference implementation of SynthID-Text [14], at the

revision pinned for this study. One layer of the tournament reweights the whole distribution,

$$p'(x) = p(x) \left(1 + g(x) - \sum_y p(y) g(y) \right), \quad (1)$$

so candidates whose bit is 1 gain probability and the rest lose it, in proportion to how much of the probability mass carries a 1. Applying this update once per layer is equivalent to the binary tournament without ever building its 2^m leaves. The sampler applies $m = 30$ layers, each with its own bits, and then draws a token from the result p_k . Figure 1a shows one layer.

Two details make the mark readable later. The first four written tokens are sampled ordinarily, which lets the verifier rebuild every later context from the output alone. When a four-token context repeats, that step is sampled ordinarily and is left out of the score, following the original construction; a repetition history of 1,024 contexts is kept for this purpose.

Averaged over fresh keyed functions, Equation 1 leaves the model’s next-token distribution unchanged; that is an identity of the construction. With one fixed key and one fixed context it does not. The sampler then draws from p_k , a calculated reweighting of p , and p_k is the law we tested the sampler against.

3.3 Reading the mark

The verifier receives the read, the key, the generation domain label, and settings fixed before any of these reads existed. It does not receive the prompt, the model, the model’s probabilities, the seed, the strand, the token grouping, or the point where the model’s output begins (Figure 1b).

It therefore searches. For each of the four window lengths L that fit the read, it tries the read as given and its reverse complement, and inside each it tries every starting base. Stepping one base at a time covers the unknown boundary and all six token groupings at once. Inside a window it regroups the DNA into tokens from that start, treats the first four tokens as context, recomputes the mark bits for the rest, and counts how many are 1. That gives S ones out of n scored bits.

Without the right key those bits are modelled as fair coin flips, so the chance probability of one window is the exact binomial tail

$$P_{\text{win}} = \Pr[\text{Binomial}(n, \frac{1}{2}) \geq S]. \quad (2)$$

The verifier scores $M = 2 \sum_L (N - L + 1)$ windows, which is 16,136 for a 3,456-base read, and reports one answer for the whole read:

$$P_{\text{read}} = \min(1, M \cdot \min_{\text{windows}} P_{\text{win}}), \quad \text{mark found} \iff P_{\text{read}} \leq \alpha. \quad (3)$$

Multiplying by M pays for every window that was tried, including overlapping ones. Nothing about the threshold was fitted to these reads: $\alpha = 0.01$ is set in advance and the correction follows from the number of windows. Equivalently, a read is called only if some window reaches strength $\log_{10}(M/\alpha) = 6.21$. Window lengths are 384, 768, 1,536 and 3,072 bases; a window whose length does not fit the read is not searched, and a read shorter than every pre-specified length is refused rather than silently rescored. Reverse-complement coordinates are mapped back onto the supplied read, and log-probabilities are retained throughout so that strong evidence is not lost to floating-point underflow.

3.4 Prompts and generation

One prompt cohort serves both models. The 256 prompts are 384-base forward-strand windows drawn from 16 accession-versioned RefSeq chromosome records across eight model organisms. Windows were chosen by hashing a fixed label, the accession, and a segment index, before any generation

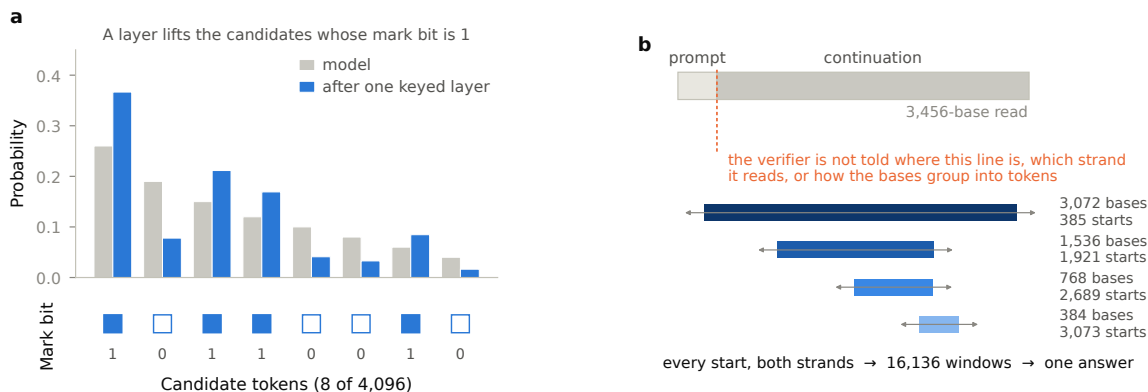


Figure 1: **Writing the mark and reading it back.** **a**, One keyed layer, drawn for eight of the 4,096 candidate tokens with an illustrative distribution. Candidates whose mark bit is 1 gain probability; the others lose it. The sampler applies 30 such layers. **b**, What the verifier sees. It gets the read and the key, and nothing else: not the prompt boundary, not the strand, not the grouping of bases into tokens. It scores every window of four lengths at every starting base on both strands—16,136 windows for a 3,456-base read—and returns one answer that has already paid for the whole search. Panel **a** illustrates Equation 1 and is not a measurement.

existed. Human, patient, controlled-access, organelle, plasmid, and ambiguous-base spans were excluded. No prompt was added, moved, or removed after any model was run.

In each model, each prompt produced two marked and two ordinary continuations of 512 tokens (3,072 bases), so each model contributes 512 marked and 512 ordinary continuations. The two draws use different replay seeds in both arms, and the two marked draws use two different keys. Both keys are published with this study so the runs can be reproduced; a deployment would use an independently generated secret key that never leaves the generating system.

3.5 Checking the sampler

For each model and prompt we fixed the model at token index 64 of its ordinary draw-zero trajectory and drew 5,000 samples per arm from that one state. Counts were compared with the exact intended law by a Monte Carlo goodness-of-fit test with 999 replicates. The marked arm was compared against p_k , not against p ; comparing it against p would test a claim the construction does not make at a fixed key. Eight further states per model were run with a deliberately broken sampler as a negative control.

3.6 Sequence measures

The model’s score is the teacher-forced negative log-likelihood per written token under the same model and token restriction used to generate, with the prompt supplied as context and excluded from the loss. Each model is scored only under itself, so the two model scores are not comparable to each other and we never compare them. Thirteen further measures were fixed before analysis: GC content, purine content, CpG content, base, dinucleotide and trinucleotide entropy, distinct 6-mer fraction, longest and mean single-base run, Jensen–Shannon drift from the prompt at $k = 1, 2, 3$, and perplexity. All 14 form one multiple-testing family under Benjamini–Hochberg correction.

3.7 Edits and statistics

Each edited read carries exactly one substituted, inserted, or deleted base inside the written part, at a position and with a replacement base drawn by a fixed random rule that does not consult the verifier. Prompts are the independent unit; the two draws from a prompt are paired repetitions. Uncertainty on the sequence measures comes from an equal-weight bootstrap over prompts with 20,000 replicates and a paired sign-flip test with 100,000 replicates. Detection intervals are exact two-sided binomial intervals at 95% over the 192 held-out prompts, and a prompt counts as positive if either of its draws was called. Window-level probabilities are exact binomial tails, and the read-level answer multiplies the smallest of them by the number of windows searched, so every P_{read} we report already covers each strand, start, and length tried. No smallest window probability is reported as a read-level result anywhere in this paper.

4 Results

4.1 Sampler fidelity: the sampler draws from the law it calculates

Before asking what the mark costs, we checked that each sampler implements the construction. Of 256 marked states, 13 fell below the nominal 0.05 level in Carbon and 5 in GENERator, against 12.8 expected by chance in each model; the ordinary arms gave 12 and 14. No arm in either model produced a rejection after correction, while all eight states run with a deliberately broken sampler were rejected in each model. Both implementations therefore behave as intended at the 256 states and 5,000 draws per state we tested, which is a statement about those states rather than a proof for every state either model can reach.

4.2 Sequence quality: the mark leaves no measurable trace

For every prompt and model we generated two marked and two ordinary continuations of 3,072 bases under the same conditions, giving 512 matched pairs from 256 prompts in each model.

In Carbon the model’s own score was 7.53386 nats per token for ordinary output and 7.54238 for marked output. The paired difference is +0.00852 nats per token, with a 95% interval from -0.04426 to $+0.05973$ and $P = 0.75$ (Figure 2a); as a standardized effect that is 0.0201. In GENERator the score was 7.80029 for ordinary output and 7.79465 for marked output, a paired difference of -0.00564 nats per token, with a 95% interval from -0.05614 to $+0.04378$, $P = 0.83$, and a standardized effect of -0.0139 . The difference points one way in Carbon and the other in GENERator, and neither interval excludes zero, which is what a construction with no detectable cost should look like twice.

The thirteen further measures, covering base composition, k -mer entropy, distinct 6-mers, single-base runs, CpG content, and drift away from the prompt’s composition, behave the same way in both models. Every one of the 14 intervals contains zero, none survives correction for multiple testing, and the smallest adjusted P -value is 0.85 in Carbon and 0.73 in GENERator. For Carbon the largest effect in absolute value is 0.084 standard deviations (Figure 2b); the equivalent summary was not retained for GENERator, so for that model we report only that no measure survives correction.

We therefore found no measurable loss in either model, which is weaker than saying there is none: the interval on each model score still allows a change of roughly ± 0.05 nats per token, and these are model scores and sequence statistics rather than measures of biological function. The two model scores are computed under different models and are not comparable to each other.

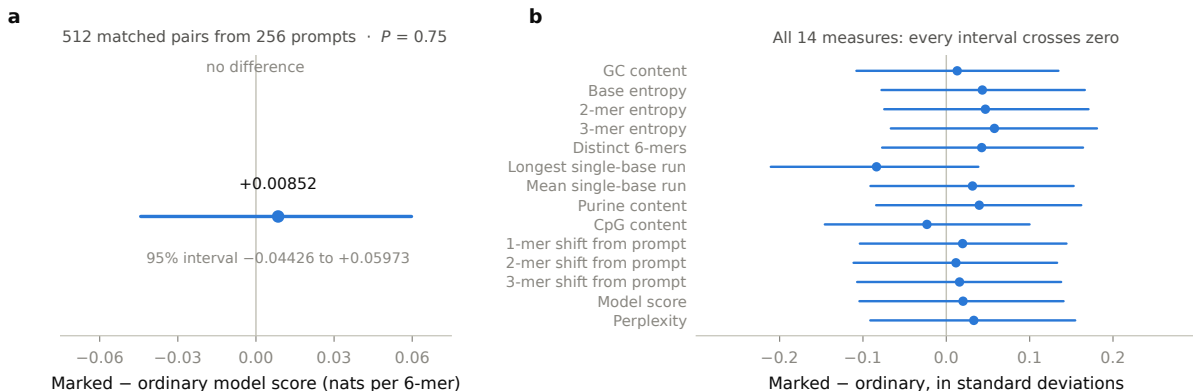


Figure 2: **The mark leaves no measurable trace, in Carbon.** All panels are Carbon; the GENERator summaries are reported in the text only. **a**, Paired difference in the model’s score between marked and ordinary continuations, over 512 matched pairs from 256 prompts. Positive means the marked continuation was slightly less probable. The bar is the 95% interval from a bootstrap that resamples prompts. **b**, All 14 pre-specified measures on one scale, as the paired difference in standard deviations, with the same interval rescaled to those units. Every interval crosses zero and none of the 14 survives correction for multiple testing.

4.3 Recall and false-positive rate: the mark is found in every marked read

We split the prompts once, before looking at any detection result: 64 prompts were used to choose between two ways of summarizing a window’s score, and 192 were held out. The threshold was not fitted on either side of that split. For the held-out prompts of each model we glued the 384-base prompt in front of each continuation, withheld the boundary, and gave the verifier the resulting 3,456-base read. Each read was checked in three ways—marked with the right key, ordinary, and marked but checked with the other draw’s key—and in four conditions: unedited, and after exactly one substituted, inserted, or deleted base. The search is the same in both models, because it depends on the read length and the window lengths rather than on the model: $M = 16,136$ windows per read, and a threshold of 6.21.

In both models the verifier found the mark in 384 of 384 marked reads in all four conditions (Table 2), a recall of 100% per model. The controls produced one positive each, and they fall in different places. In Carbon one of 384 ordinary reads passed and no wrong-key read did; in GENERator no ordinary read passed and one of 384 wrong-key reads did, in the unedited, substituted, and inserted conditions but not the deleted one. Each of those single positives is a false-positive rate of 0.26% in its cell. We report recall and the two false-positive rates separately rather than a pooled accuracy, because how many marked, ordinary, and wrong-key reads a cohort contains is a design choice, and a pooled figure would move with that choice rather than with the verifier.

For Carbon the separation is not marginal. On unedited reads the weakest marked read reached window strength 124.8, while the strongest ordinary read reached 6.60 and the strongest wrong-key read reached 5.69, against a threshold of 6.21 (Figure 3a). The single ordinary read that passed sits just above the line, which is what a correctly calibrated test at $\alpha = 0.01$ should occasionally produce. The window-strength distribution was not retained for GENERator, so this margin, and the edit-by-edit strengths below, are Carbon results; for GENERator we have the decisions but not the distances behind them.

Because the two draws from a prompt are not independent, we count prompts. In each model

Table 2: Reads in which the verifier found the mark, for each model, on 192 held-out prompts with two draws each. Each edit is exactly one base. The last two rows of each block count prompts rather than reads, because the two draws from a prompt are paired repetitions; intervals are exact and two-sided. Nothing is pooled across the two models.

Edit applied to the read	Marked, right key	Ordinary	Marked, wrong key
<i>Carbon</i>			
None	384/384	1/384	0/384
One substitution	384/384	1/384	0/384
One insertion	384/384	1/384	0/384
One deletion	384/384	1/384	0/384
Prompts with a positive draw	192/192	1/192	0/192
Exact 95% interval	98.10–100%	0.013–2.868%	0–1.903%
<i>GENERator</i>			
None	384/384	0/384	1/384
One substitution	384/384	0/384	1/384
One insertion	384/384	0/384	1/384
One deletion	384/384	0/384	0/384
Prompts with a positive draw	192/192	0/192	1/192
Exact 95% interval	98.10–100%	0–1.903%	0.013–2.868%

all 192 prompts had both marked draws found, giving an exact 95% interval of 98.10–100%. In Carbon one prompt of 192 produced an ordinary positive, giving 0.013–2.868%, and none produced a wrong-key positive, giving an upper bound of 1.903% (Figure 3b). In GENERator the same two numbers appear in the opposite columns: no prompt produced an ordinary positive, upper bound 1.903%, and one produced a wrong-key positive, 0.013–2.868%. In both models the repeated control positive in Table 2 is one prompt and draw counted more than once, not several events. Carbon’s ordinary positive survived all four conditions because its edit fell outside the window carrying its evidence; GENERator’s wrong-key positive survived the substitution and the insertion but not the deletion.

For GENERator we also have a direct check of the null the read-level answer is built on. Scoring ordinary output with the token grouping supplied, over the full 256-prompt cohort, the number of prompts producing a positive was 2, 6, 6 and 4 at the four window lengths, against about five expected at each, with exact two-sided P -values of 0.25, 0.87, 0.76 and 0.84. The analytic null therefore fits ordinary output at every length we checked. This check was run for GENERator only.

4.4 Edit robustness: one edited base does not hide the mark

The mechanism here was measured in Carbon. A substitution changes the mark bits only near itself, and the evidence barely moves: the median window strength falls from 790.6 on unedited reads to 774.7 (Figure 4a). An insertion or a deletion is different in kind. It shifts every downstream base into a different token grouping, so the long window that carried the evidence before the edit is now half correct and half scrambled.

The search absorbs this by itself. On unedited reads the full 3,072-base window carried the strongest evidence in 383 of 384 reads. After an insertion it did so in only 203, with 181 reads best explained by a shorter window; after a deletion, 232 and 152 (Figure 4b). A window that starts

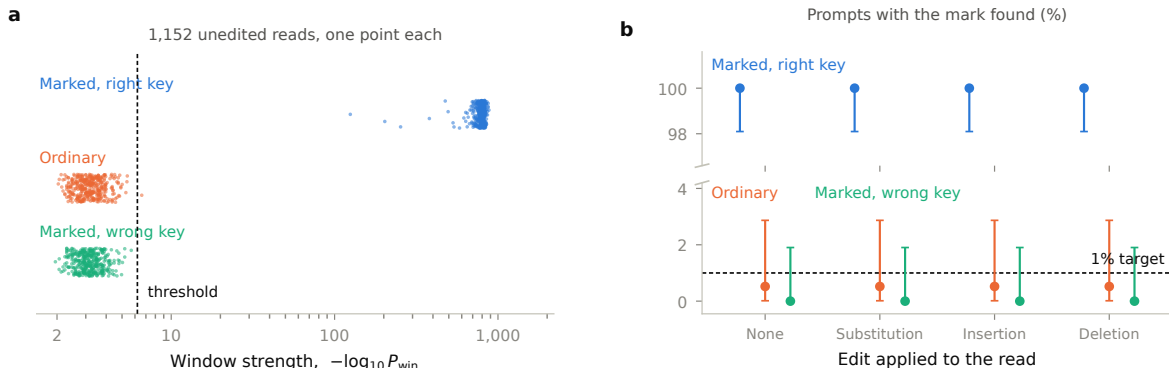


Figure 3: **Finding the mark without being told where the DNA starts, in Carbon.** All panels are Carbon; GENERator’s decisions are in Table 2, and its window strengths were not retained. **a**, Strength of the best window in each unedited read, one point per read, on a log scale. The dashed line is the threshold a read must clear once the search over 16,136 windows has been paid for. Marked reads sit two to three orders of magnitude past it; controls sit below it, with one ordinary read just above. **b**, Share of the 192 held-out prompts in which the mark was found, with exact 95% intervals, split across a break in the axis. Results are the same in all four edit conditions. The control intervals reach above the 1% target, so these data are consistent with a false-positive rate below 1% without establishing one.

after the edit and stops before the read runs out recovers the correct grouping, and the verifier finds it because it tried every start. The median strength falls to 431.5 for an insertion and 443.7 for a deletion, and the weakest read in any condition still reached 82.2 against a threshold of 6.21. Every marked read was still found.

GENERator supplies the outcome but not the mechanism, because its window strengths and winning window lengths were not retained: every one of its 384 marked reads was found in all four conditions, and its one wrong-key control positive disappeared after the deletion. So the claim that the search repairs a frame shift rests on Carbon, and the claim that detection survives one edit rests on both.

5 Discussion

The verifier in these experiments works without the things a detector is usually assumed to have. It is not told the prompt, the model, the strand, the token grouping, or where the model’s output begins, and it still found every marked read in this corpus, in both models. The cost of that freedom is explicit rather than hidden: the answer is multiplied by the number of windows tried, so a larger search weakens every claim it might support. What makes the approach practical is that the mark, spread over 3,072 written bases, has evidence far in excess of what the threshold demands, so paying for 16,136 windows costs a factor that the signal does not notice. This is the main thing the numbers say, and it is also the main thing that will not transfer automatically: a short generated stretch would spend most of its evidence on the search alone, and a longer read would pay a larger correction for the same signal, so the length at which position-independent detection stops working is a real quantity that this study does not measure.

Running one implementation through two models of different size, training corpus, and tokenizer

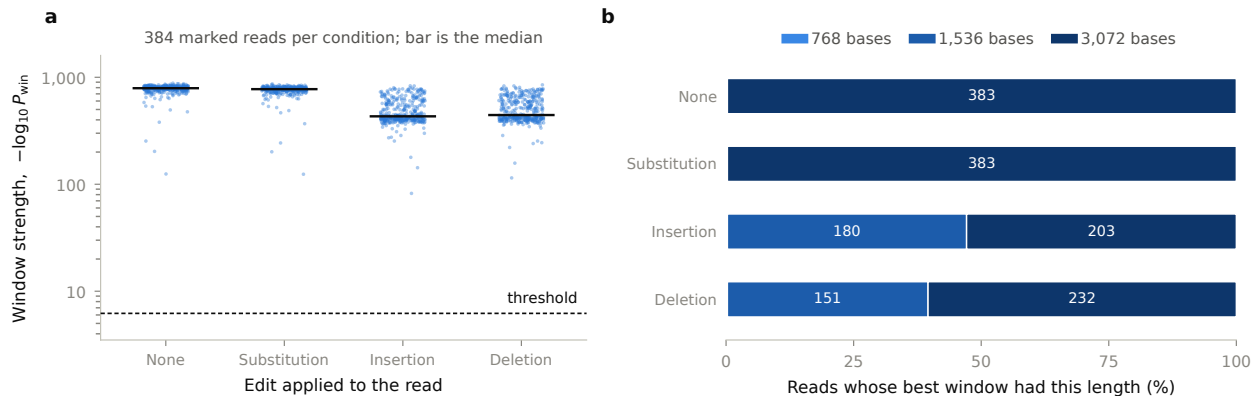


Figure 4: **What one edited base does, in Carbon.** All panels are Carbon. **a**, Strength of the best window in each of the 384 marked reads per condition, on a log scale, with the median as a bar. The dashed line is the threshold. A substitution costs almost nothing; an insertion or deletion costs about half the evidence and still leaves more than an order of magnitude of margin. **b**, Length of the window that carried the strongest evidence. Numbers inside the bars are read counts out of 384. Unedited and substituted reads are almost always best explained by the full 3,072-base window; after an insertion or deletion, a shorter window that starts past the edit often wins.

construction is what makes the result about the construction rather than about a model. The detector’s arithmetic does not change between them, because it depends on the read and the window lengths rather than on what wrote the read, and the outcome did not change either: full recall in every condition, one control positive per model, and no measurable model-score cost in either. What two models cannot tell us is which of them writes better DNA, or whether a third model with a different tokenisation would behave the same way. Portability of an implementation is a weaker and more useful claim than equivalence of models, and it is the one we make.

The behaviour under insertion and deletion is worth separating from ordinary robustness. This part was measured in Carbon alone. A substitution is a local event and the count barely moves. An insertion or deletion is not local at all in a model that writes six bases at a time, because it slides every downstream base into a different word and destroys the grouping that the long window depended on. The evidence does drop by roughly half, but detection does not fail, and the reason is visible in which window wins: after an indel the strongest evidence usually comes from a shorter window that starts past the edit, where the grouping has recovered. The search that made the problem expensive is the same mechanism that repairs it. That is a property of searching every start, not a general robustness guarantee, and it says nothing about a read carrying many edits. It is also worth being clear that no watermark survives an adversary who can rewrite a sequence at quality; robustness claims of this kind are always claims about a stated edit budget [30], and ours is one substitution, insertion, or deletion applied by a rule that never looks at the verifier.

Coding theory has treated this asymmetry as the harder case for a long time, because an insertion or a deletion destroys the index rather than one symbol, and the standard remedy is to restore the index explicitly: indexing a stream with a synchronisation string turns synchronisation errors back into ordinary substitution errors, so a conventional code can finish the job [15]. The cryptographic counterpart is a pseudorandom code, whose codewords stay indistinguishable from random while remaining decodable under substitutions and deletions, which is the property a watermark on an editable channel wants [5]. Our verifier buys the same synchronisation by brute

force instead. That needs no change to the sampler and no redundancy in the sequence, and it pays for itself in the size of the search rather than in the construction.

Complementary work marks designed coding sequences through synonymous substitution and carries the mark into the translated protein [31]; that approach assumes a coding region and a design step, and it answers a question about function preservation that ours does not. The two are naturally combined, and the combination is a better description of what a provenance system for synthetic DNA would eventually need than either alone.

6 Limitations

The scope is two models, one watermark, one verifier, and one prompt cohort, and each model contributes a single execution whose confirmatory replication is still outstanding: nothing above has been reproduced independently of the run that produced it. Both models write six-base tokens, as do several other current genomic models [27], which is what makes the grouping problem general rather than particular; but models that read DNA one base at a time face a different version of it, and models with much longer context would change the arithmetic of the search. The two models also share their prompts, so they replicate the implementation across models and not across corpora.

The two runs are unequal in what they recorded, and the paper is only as strong as the weaker side in each place. Carbon carries the window-strength separation, the effect sizes of the 14 sequence measures, and the winning window length under each edit; GENERator carries the aligned-detector null fit. Where a claim rests on one model we have said so, and the four figures are Carbon throughout.

The execution itself falls short of the standard this study sets for itself. Generation ran on GPU hardware and detection on x86-64 CPUs, not on the single documented machine our replication plan names for every result that reaches a paper, so hardware is a difference between these numbers and their intended replication rather than a controlled variable. For the Carbon detection run, the retained protocol document does not match the protocol hash recorded in the result, a discrepancy recorded when the result was reviewed rather than resolved. Neither problem can change a count in Table 2, both are reasons to treat these numbers as one execution rather than a settled measurement, and the replication that would settle them is the immediate next step and not part of this paper.

The false-positive behaviour is consistent with the 1% target in both models but does not establish it. In Carbon one ordinary prompt of 192 passed and no wrong-key prompt did; in GENERator one wrong-key prompt passed and no ordinary prompt did. Both patterns are what a correctly calibrated test should produce, and neither resolves a rate near 1%, because 192 independent prompts cannot: the interval around a single positive prompt runs to 2.87% and the upper bound on a zero count is 1.90%, in either model and in either control family. Pinning the rate down needs more independent prompts, not a different verifier.

Neither is any of this a security result, because the keys used here are published so that the study can be reproduced, and a published key cannot test whether a deployed secret key could be recovered from many observed outputs; key secrecy is an assumption we make rather than a finding, as is the standard modelling assumption that the mark bits behave like fair coin flips to anyone without the key. In text, keys of the distortion-free family have been recovered by adaptive prompting [25] and watermark rules reverse-engineered by querying the watermarked model [19]; whether either transfers to a 4,096-token genomic vocabulary is a question this study does not ask, and the attacker assumed throughout neither holds the key nor sees the verifier’s score.

And none of it is a biological result: the model’s score and the 14 sequence measures are

statistics about letters, they do not measure whether the DNA folds, expresses, replicates, or is safe, and a watermark that leaves those statistics unchanged has not thereby been shown to leave function unchanged. That last gap is the one most worth closing, and closing it needs assays rather than more sequence statistics.

7 Conclusion

Adding the SynthID tournament watermark to two genomic language models cost nothing we could measure in either model’s own likelihood or in 14 pre-specified sequence measures. A verifier holding only the DNA and the key found the watermark in every one of 384 held-out marked reads per model, unedited and after any single substituted, inserted, or deleted base, while paying in full for a search over both strands, every starting base, and four window lengths. Unmarked and wrong-key reads behaved as the test predicts, one control positive per model, though 192 prompts are too few to place a tight bound near a 1% false-positive rate. The result that matters beyond this corpus is structural: when a generated stretch is long enough, the statistical price of not knowing where it begins is small, and the same exhaustive search that makes the problem hard is what repairs the frame shift an inserted or deleted base causes.

Data and code availability

The two keys used in these experiments are public fixture material published with this study, so the runs can be reproduced; they are not deployment secrets, and no result here tests whether a deployed secret key could be recovered. The generated sequences, the verifier outputs underlying every figure, the prompt set, and the analysis and figure code are available from the authors on request.

References

- [1] Scott Aaronson and Hendrik Kirchner. Watermarking GPT outputs. Talk slides, 2022. <https://www.scottaaronson.com/talks/watermark.ppt>; the source usually credited for exponential-minimum, or Gumbel, keyed sampling.
- [2] David Baker and George Church. Protein design meets biosecurity. *Science*, 383(6681):349, 2024.
- [3] Loubna Ben Allal, Qiuyi Li, Maurizio Fiusco, Lewis Tunstall, Kashif Rasul, Edward Beeching, Dana Aubakirova, Carlos Patiño, Thibaud Frere, Anton Lozhkov, Georgia Channing, Thomas Wolf, Diego di Bernardo, and Leandro von Werra. Carbon: decoding the language of life. *bioRxiv*, 2026. Posted 25 May 2026.
- [4] Yanshuo Chen, Zhengmian Hu, Yihan Wu, Ruibo Chen, Yongrui Jin, Marcus Zhan, Chengjin Xie, Wei Chen, and Heng Huang. Enhancing privacy in biosecurity with watermarked protein design. *Bioinformatics*, 41(7):btaf141, 2025.
- [5] Miranda Christ and Sam Gunn. Pseudorandom error-correcting codes. In *Advances in Cryptology — CRYPTO 2024*, 2024. IACR ePrint 2024/235; robust to substitutions and deletions.

- [6] Miranda Christ, Sam Gunn, and Or Zamir. Undetectable watermarks for language models. In *Proceedings of the 37th Annual Conference on Learning Theory (COLT)*, volume 247 of *PMLR*, pages 1125–1139, 2024. arXiv:2306.09194.
- [7] Hugo Dalla-Torre, Liam Gonzalez, Javier Mendoza-Revilla, Nicolas Lopez Carranza, Adam Henryk Grzywaczewski, Francesco Oteri, Christian Dallago, Evan Trop, Bernardo P. de Almeida, et al. Nucleotide Transformer: building and evaluating robust foundation models for human genomics. *Nature Methods*, 22:287–297, 2025.
- [8] Sumanth Dathathri, Abigail See, Sumedh Ghaisas, Po-Sen Huang, Rob Chen, Nenad Kushman, Ilia Shumailov, Eugene Tarassov, Pushmeet Kohli, et al. Scalable watermarking for identifying large language model outputs. *Nature*, 634(8035):818–823, 2024.
- [9] Abdulrahman Diaa, Toluwani Aremu, and Nils Lukas. Optimizing adaptive attacks against watermarks for language models. In *Proceedings of the 42nd International Conference on Machine Learning (ICML)*, volume 267 of *PMLR*, pages 13546–13569, 2025.
- [10] Jaiden Fairuze, Sanjam Garg, Somesh Jha, Saeed Mahloujifar, Mohammad Mahmood, and Mingyuan Wang. Publicly-detectable watermarking for language models. *IACR Communications in Cryptology*, 1(4), 2025.
- [11] GenerTeam. GENERator-v2-eukaryote-1.2b-base model card and released modeling code. <https://huggingface.co/GenerTeam/GENERator-v2-eukaryote-1.2b-base>, 2026. Revision c41b0018; `modeling_generator.py` defines `_BPLogitsProcessor`, which samples each of six bases independently inside `generate()`.
- [12] Daniel G. Gibson, John I. Glass, Carole Lartigue, Vladimir N. Noskov, Ray-Yuan Chuang, Mikkel A. Algire, Gwynedd A. Benders, Michael G. Montague, Li Ma, et al. Creation of a bacterial cell controlled by a chemically synthesized genome. *Science*, 329(5987):52–56, 2010.
- [13] Thibaud Gloaguen, Nikola Jovanović, Robin Staab, and Martin Vechev. Discovering spoofing attempts on language model watermarks. In *Proceedings of the 42nd International Conference on Machine Learning (ICML)*, volume 267 of *PMLR*, pages 19562–19584, 2025.
- [14] Google DeepMind. SynthID-Text reference implementation. <https://github.com/google-deepmind/synthid-text>, 2024. Apache-2.0; revision addb4a15. Source of the tournament formula, default layer depth, context width, and repeated-context masking used here.
- [15] Bernhard Haeupler and Amirbehshad Shahrashbi. Synchronization strings: Codes for insertions and deletions approaching the singleton bound. In *Proceedings of the 49th Annual ACM SIGACT Symposium on Theory of Computing (STOC)*, 2017.
- [16] Dominik Heider and Angelika Barnekow. DNA-based watermarks using the DNA-Crypt algorithm. *BMC Bioinformatics*, 8:176, 2007.
- [17] Zhengmian Hu, Lichang Chen, Xidong Wu, Yihan Wu, Hongyang Zhang, and Heng Huang. Unbiased watermark for large language models. In *International Conference on Learning Representations (ICLR)*, 2024.
- [18] HuggingFaceBio. Carbon-500M model card. <https://huggingface.co/HuggingFaceBio/Carbon-500M>, 2026. Main revision 9796b752 is a standard causal LM; the fns revision 974bc8a5 adds base-pair-level generation and scoring.

- [19] Nikola Jovanović, Robin Staab, and Martin Vechev. Watermark stealing in large language models. In *Proceedings of the 41st International Conference on Machine Learning (ICML)*, volume 235 of *PMLR*, 2024.
- [20] John Kirchenbauer, Jonas Geiping, Yuxin Wen, Jonathan Katz, Ian Miers, and Tom Goldstein. A watermark for large language models. In *Proceedings of the 40th International Conference on Machine Learning (ICML)*, volume 202 of *PMLR*, pages 17061–17084, 2023. arXiv:2301.10226.
- [21] Rohith Kuditipudi, John Thickstun, Tatsunori Hashimoto, and Percy Liang. Robust distortion-free watermarks for language models. *Transactions on Machine Learning Research*, 2024. arXiv:2307.15593.
- [22] Qiuyi Li, Zhihao Zhan, Shikun Feng, Yiheng Zhu, Yuan He, Wei Wu, Zhenghang Shi, Shengjie Wang, Zongyong Hu, Zhao Yang, Jiaoyang Li, Jian Tang, Haiguang Liu, and Tao Qin. GENERator-v2: Reconciling coarse tokenization with single-nucleotide resolution in genomic language modeling. *bioRxiv*, 2026. Version 2.
- [23] Michael Liss, Daniela Daubert, Kathrin Brunner, Kristina Kliche, Ulrich Hammes, Andreas Leiherer, and Ralf Wagner. Embedding permanent watermarks in synthetic genes. *PLoS ONE*, 7(8):e42465, 2012.
- [24] Eric Nguyen, Michael Poli, Matthew G. Durrant, Brian Kang, Dhruva Katrekar, David B. Li, Liam J. Bartie, Armin W. Thomas, Samuel H. King, Garyk Bixi, et al. Sequence modeling and design from molecular to genome scale with Evo. *Science*, 386(6723):eado9336, 2024.
- [25] Shayleen Reynolds, Hengzhi He, Dung Daniel T. Ngo, Saheed Obitayo, Niccolò Dalmasso, Guang Cheng, Vamsi K. Potluru, and Manuela Veloso. Breaking distortion-free watermarks in large language models, 2025.
- [26] Vinu Sankar Sadasivan, Aounon Kumar, Sriram Balasubramanian, Wenxiao Wang, and Soheil Feizi. Can AI-generated text be reliably detected? stress testing AI text detectors under various attacks. *Transactions on Machine Learning Research*, 2025.
- [27] Wei Wu, Qiuyi Li, Yuanyuan Zhang, Zhihao Zhan, Ruipu Chen, Mingyang Li, Kun Fu, Junyan Qi, Yongzhou Bao, Chao Wang, et al. GENERator: A long-context generative genomic foundation model. *arXiv preprint arXiv:2502.07272*, 2025.
- [28] Yihan Wu, RuiBo Chen, Zhengmian Hu, Yanshuo Chen, Junfeng Guo, Hongyang Zhang, and Heng Huang. Distortion-free watermarks are not truly distortion-free under watermark key collisions, 2024.
- [29] Yihan Wu, Zhengmian Hu, Junfeng Guo, Hongyang Zhang, and Heng Huang. A resilient and accessible distribution-preserving watermark for large language models. In *Proceedings of the 41st International Conference on Machine Learning (ICML)*, volume 235 of *PMLR*, 2024.
- [30] Hanlin Zhang, Benjamin L. Edelman, Danilo Francati, Daniele Venturi, Giuseppe Ateniese, and Boaz Barak. Watermarks in the sand: Impossibility of strong watermarking for generative models. In *Proceedings of the 41st International Conference on Machine Learning (ICML)*, volume 235 of *PMLR*, pages 58851–58880, 2024. arXiv:2311.04378.
- [31] Zaixi Zhang, Ruofan Jin, Le Cong, and Mengdi Wang. Securing the language of life: Inheritable watermarks from DNA language models to proteins. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2025. arXiv:2509.18207.